IDMC MS4 Milestone Assessment

Topic-wise Study Guide for the 160-MCQ Practice Bank

Every one of the 160 questions is mapped to a topic, tagged by difficulty, and reduced to a one-line "cue → answer" pair. Learn the 29 topic rules and you have covered the whole bank.

How to use this guide

- **Memorise concepts, not questions.** Easy questions are the same concept asked 4 times in different words, so one rule per topic covers all of them. Option letters are shuffled between twins (the same answer is A in one and D in another), so never memorise letters.
- **Marks are weighted toward Medium and Hard.** Easy = 100 marks, but Medium (30 Qs × 2) + Hard (30 Qs × 2) = 120 marks. 60 questions carry more marks than the 100 Easy ones, so give them real time.
- **Use the ✓column.** Each topic table has a tick box per question. Cover the Answer column, answer from the cue, then tick only when you were right. Zero unticked boxes = nothing missed.
- **Read the trap line.** Wrong options in this bank repeat the same patterns (wrong transformation, "hardcode", "duplicate", "ignore", "increase compute"). Spotting them eliminates 2–3 options fast.
- **Note on the file name.** The upload is named "MS3" but the question bank itself is titled MS4. This guide follows the content of the bank.

1. Question Bank Snapshot

Difficulty	Questions	Marks each	Total marks	Question numbers
Easy	100	1	100	Q1 – Q100
Medium	30	2	60	Q101 – Q130
Hard	30	2	60	Q131 – Q160
Total	**160**	**–**	**220**	**Q1 – Q160**

Topic-wise distribution (all 29 topics)

#	Topic	E	M	H	Tot.	Question numbers (E = Easy · M = Medium · H = Hard)
A. Data Quality & Data Shape						
1	**Data Profiling**	4	1	1	**6**	E: 1–4 · M: 101 · H: 147
2	**Error Quarantine & Restart**	–	1	1	**2**	M: 128 · H: 142
3	**Hierarchical Data (JSON Flattening)**	–	1	1	**2**	M: 116 · H: 139
B. Mapping Transformations						
4	**Expression Transformation**	4	1	–	**5**	E: 5–8 · M: 130
5	**Filter Transformation**	4	–	–	**4**	E: 9–12
6	**Joiner Transformation**	4	–	–	**4**	E: 13–16
7	**Aggregator Transformation**	4	1	–	**5**	E: 17–20 · M: 104
8	**Router Transformation**	4	1	–	**5**	E: 21–24 · M: 105
9	**Lookup Transformation & Caching**	4	1	2	**7**	E: 25–28 · M: 106 · H: 134, 150
10	**Sorter Transformation**	4	1	–	**5**	E: 29–32 · M: 122
11	**Union Transformation**	4	1	–	**5**	E: 33–36 · M: 123
12	**Rank Transformation**	4	1	–	**5**	E: 37–40 · M: 124
C. Reuse & Configuration						
13	**Mapplets & Reusable Logic**	4	1	2	**7**	E: 41–44 · M: 102 · H: 135, 151
14	**Mapping vs Mapping Task**	4	1	–	**5**	E: 45–48 · M: 120

#	Topic	E	M	H	Tot.	Question numbers (E = Easy · M = Medium · H = Hard)
15	**Parameterization**	4	1	2	**7**	E: 61–64 · M: 107 · H: 137, 145
16	**Connections**	4	1	1	**6**	E: 57–60 · M: 119 · H: 148
D. Runtime Infrastructure						
17	**Secure Agent**	4	1	1	**6**	E: 49–52 · M: 117 · H: 136
18	**Runtime Environment**	4	1	–	**5**	E: 53–56 · M: 118
E. Taskflow Orchestration						
19	**Sequential Taskflow**	4	–	–	**4**	E: 65–68
20	**Parallel Taskflow**	4	2	2	**8**	E: 69–72 · M: 109, 129 · H: 133, 144
21	**Decision Step (Conditional Branching)**	4	1	1	**6**	E: 73–76 · M: 108 · H: 149
22	**Fault Handling**	4	1	2	**7**	E: 77–80 · M: 110 · H: 138, 159
23	**Command & Event Steps (Triggers/Listeners)**	4	2	–	**6**	E: 81–84 · M: 126, 127
F. Data Movement Patterns						
24	**Change Data Capture (CDC) / Incremental**	4	1	1	**6**	E: 85–88 · M: 111 · H: 132
25	**Batch vs Real-Time Ingestion**	4	1	1	**6**	E: 93–96 · M: 112 · H: 146
G. Performance & Observability						
26	**SQL Pushdown**	4	2	4	**10**	E: 89–92 · M: 103, 121 · H: 131, 141, 154, 160
27	**Monitoring & Performance Diagnosis**	–	2	2	**4**	M: 113, 125 · H: 140, 152
28	**Snowflake Context Questions**	–	–	4	**4**	H: 155–158
H. CAI & CAI-CDI Integration						
29	**CAI Connectors & CAI-CDI Integration**	4	2	2	**8**	E: 97–100 · M: 114, 115 · H: 143, 153
	TOTAL	**100**	**30**	**30**	**160**	*Every question Q1–Q160 appears exactly once*

2. One-Page Cheat Sheet: Cue → Answer

If you only have 20 minutes, learn this table. Spot the cue in the question, give the answer.

Topic	Spot it when the question says…	Answer
Data Profiling	New or unknown feed; nulls; inconsistent formats; no data dictionary; unexpected value patterns	**Profile the source data first, then design rules from the findings**
Error Quarantine & Restart	Some invalid records; missing mandatory fields; valid rows must continue; rejects need investigation	**Quarantine/error flow for rejects; valid rows continue; controlled reprocessing**
Hierarchical Data (JSON Flattening)	Nested arrays / JSON; relational output needed in Snowflake; parent-child relationships	**Flatten to relational shape and preserve parent identifiers/relationships**
Expression Transformation	Calculate / derive / standardise / convert a field per row	**Expression transformation**
Filter Transformation	"Only rows where…" one condition; drop the rest	**Filter transformation**
Joiner Transformation	Two pipelines/sources matched by a business key into one output row	**Joiner with the required matching condition**
Aggregator Transformation	Total / count / average / max per group; one row per group	**Aggregator with the required group fields**
Router Transformation	Split rows into several categories/paths; repeated filter branches	**Router with separate output-group conditions**
Lookup Transformation & Caching	Enrich rows with reference attributes by key; repeated reference access; cache size/refresh	**Lookup; cache small stable data; refresh cache per batch; re-evaluate (pushdown) when reference is huge**
Sorter Transformation	Rows must be delivered in a specific order	**Sorter with the required sort keys**
Union Transformation	Same-structure feeds merged into one stream; no business-key match	**Union to append compatible row streams**
Rank Transformation	Top N / bottom N / best / worst records	**Rank configured for the required top/bottom records**
Mapplets & Reusable Logic	Same standardise/validate steps repeated in many mappings; shared rule drift; shared component change risk	**Reusable mapplet; common stable rules inside, client-specific outside; managed new version for risky changes**
Mapping vs Mapping Task	Design vs execution settings; one mapping, different schedules/parameters	**Logic in the mapping; runtime config in mapping task(s)**
Parameterization	Different paths/tables/connections per environment, tenant or run; avoid copies	**Parameterize environment/tenant-specific values; keep one mapping design**
Connections	Host/user/auth details for a system; same system used by many mappings; credential sprawl	**Centrally managed connection object referenced by mappings/tasks**
Secure Agent	On-prem / private network source; job execution; runtime availability	**Secure Agent (several in one runtime environment for resilience)**
Runtime Environment	Which agents/group will execute this task; restrict to approved agents	**Runtime environment containing the Secure Agent resources**
Sequential Taskflow	Strict dependencies; step needs the previous step's result; no overlap allowed	**Sequential taskflow execution**
Parallel Taskflow	Independent branches; batch window/SLA too long; final step needs all branches	**Parallel independent branches, then gate the dependent step (hybrid)**
Decision Step (Conditional Branching)	Behaviour depends on a result: pass/fail, warning/critical, row count	**Decision step with a branch per outcome**
Fault Handling	Temporary failures; recovery path; some failures critical, others not; restart point	**Taskflow fault handling (per path) with retry/recovery; idempotent/checkpointed steps**
Command & Event Steps (Triggers/Listeners)	Shell utility inside orchestration; start on an event at unpredictable time	**Taskflow command step / trigger or listener**
Change Data Capture (CDC) / Incremental	Unchanged rows reloaded; only changed records needed; failed run and watermark	**CDC / incremental processing; advance checkpoint only after success**
Batch vs Real-Time Ingestion	Nightly/monthly files vs continuous events; latency tolerance	**Batch for scheduled files; real-time for continuous low-latency events**
SQL Pushdown	Heavy data movement; engine-side joins/aggregations; Snowflake compute available; filter late	**Pushdown compatible work to DB; refactor/partial pushdown where blocked**
Monitoring & Performance Diagnosis	Intermittent failures; slowdowns; unpredictable spikes; unknown slow step	**Monitoring + execution logs, correlate evidence, then targeted optimisation**
Snowflake Context Questions	Snowflake queue/concurrency; idle warehouse cost; repeated identical queries; many micro-partitions scanned	**Multi-cluster · Auto-suspend · Result cache (only if data unchanged) · Clustering**
CAI Connectors & CAI-CDI Integration	External API call; service request starting a CDI mapping; API timeout; reuse across processes	**CAI service connector + integrated CDI invocation; async with trackable status; reusable connector**

3. Look-alike Pairs & Elimination Patterns

3.1 Look-alike concepts: tell them apart

Confusable set	The one-line difference
Joiner vs Union vs Lookup	Joiner = two pipelines matched on a key (adds columns). Union = same-structure streams stacked (adds rows, no key). Lookup = fetch attributes from a reference object by key.
Filter vs Router vs Rank	Filter = keep rows passing ONE condition. Router = split into SEVERAL named groups. Rank = keep only top/bottom N by a measure.
Aggregator vs Rank vs Sorter	Aggregator = group totals, one row per group. Rank = top/bottom N rows with detail kept. Sorter = only changes order, never reduces rows.
Expression vs Aggregator	Expression = per-row calculation. Aggregator = calculation across rows of a group.
Mapping vs Mapping Task	Mapping = transformation design (what). Task = runnable config: parameters, runtime environment, schedule (how / where / when).
Connection vs Secure Agent vs Runtime Environment	Connection = stored endpoint + credentials. Secure Agent = the runtime engine that executes and reaches systems. Runtime environment = the group of agents a task is assigned to.
Sequential vs Parallel	Sequential = dependency on previous result. Parallel = independent branches. A dependent final step is gated until all branches finish (hybrid).
Decision step vs Fault handling	Decision = branch on a normal runtime outcome (pass/fail, warning, row count). Fault handling = controlled response to errors (retry, alternate path, notify).
Command step vs Trigger/Listener	Command = run a system-level action inside a taskflow. Trigger/Listener = start a taskflow when an event arrives.
Batch vs Real-time	Batch = scheduled, grouped, latency-tolerant. Real-time = continuous, low latency.
CDC vs full reload	CDC = only inserts/updates/deletes since last checkpoint. Full reload = everything every time (wrong when volume grows and few rows change).
Pushdown vs scaling the agent	Pushdown = move work to the data (less movement). Scaling = more engine resources without fixing movement; usually a wrong answer.

3.2 The "always wrong" answer blacklist

Across all 160 questions the incorrect options keep reusing these patterns. If an option looks like one of them, eliminate it.

Pattern	Typical wrong options
Wrong tool for the job	Sorter calculating totals, Union filtering rows, Lookup/Joiner running commands, Router orchestrating tasks, mapplet storing credentials.
Hardcode / copy / duplicate	Hardcode environment values, copy mappings per environment/tenant/schedule, duplicate processes per error.
"One new Secure Agent per…"	Creating a Secure Agent per copy of logic, per parameter value or per tenant.
Ignore, hide or skip	Ignore failures, disable monitoring/logging, discard rejected rows without trace, skip profiling.
Blunt-force scaling	Increase memory/compute/warehouse size without reviewing execution evidence.
All-or-nothing	Stop whole pipeline on first bad row; disable all pushdown because one rule is incompatible; force full pushdown or change the business rule.
Manual handoff / exposure	Operator starts the job manually after CAI; expose the database publicly instead of using an agent.
Full reload / extra movement	Reload everything each cycle; extract all history then filter later; move more data into the engine.
Losing relationships	Flatten JSON without parent identifiers; discard nested arrays.

3.3 Hard-tier answer signatures

Correct Hard answers are usually the precise, qualified option, not the extreme one. Look for words like only, then, per, preserved, after, evaluate.

If the scenario says…	The best answer…	Question
Some expressions block pushdown	**Refactor ONLY the incompatible expressions**	Q131
Complex expression inside join condition	**Derive compatible key BEFORE the join**	Q141
One rule cannot be pushed	**PARTIAL pushdown, keep that rule in engine**	Q160
Predicate supported by source DB	**Push the date predicate to the source**	Q154
Failure before watermark advanced	**Advance checkpoint ONLY AFTER success; restart from last committed**	Q132
Independent branches + final dependency	**Parallel, THEN gate the final step**	Q133, Q144
Services of different criticality	**Fault handling PER service path**	Q138
Mid-flow failure, avoid double ingestion	**Fault handling + idempotent/checkpointed steps**	Q159
Pass / warning / critical outcomes	**Decision logic with DISTINCT branches**	Q149
API times out while CDI runs long	**ASYNC trigger with trackable status**	Q143
Many processes call the same API	**Reusable service connector; process logic stays separate**	Q153
Shared logic + client-specific rules	**Stable shared rules in mapplet; client rules OUTSIDE**	Q135
Shared component change risks consumers	**Managed NEW VERSION, migrate after validation**	Q151
Reference data grew massively	**EVALUATE pushdown / DB-side join**	Q150
Large lookup refreshed daily	**Cache REFRESHED EACH BATCH**	Q134
Nested arrays	**Flatten WITH parent identifiers**	Q139
Environment promotion, secrets	**PARAMETERIZE + approved runtime config**	Q137
Many tenants, same logic	**Parameters; ONE transformation design**	Q145
Single agent is single point of failure	**Multiple agents in one runtime environment**	Q136
Spikes with no clear cause	**CORRELATE evidence BEFORE targeted tuning**	Q140
Delays hidden by final status	**STEP-LEVEL monitoring and logs**	Q152
Rejects must be traceable and reprocessable	**Quarantine path + controlled reprocessing**	Q142
Sensitive credentials spread across notes/files	**Central IDMC connection objects**	Q148
Snowflake: concurrency / idle cost / cache / pruning	**Multi-cluster / auto-suspend / result cache if unchanged / clustering**	Q155–Q158

4. Topic-by-Topic Study Sheets

For each topic: the rule to memorise, a memory hook, the wrong-answer traps, then every question with its cue and answer. Cover the Answer column and self-test.

Group A | Data Quality & Data Shape

1. Data Profiling 6 Qs (4 Easy · 1 Medium · 1 Hard)

RULE Profile the source FIRST. Profiling shows nulls, patterns, distinct values, completeness and distributions, and those findings drive the cleansing and validation rules. Never design rules on assumptions.

HOOK *Look before you cleanse.*

TRAPS Increase Secure Agent memory · build a schedule first · define target load order first · start transformations using assumed formats · skip profiling and rely on field names · one global rule with no source analysis.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q1**	Easy	Customer feed with undocumented nulls and inconsistent formats; what first?	**Profile source before designing transformation rules**
☐	**Q2**	Easy	Mapping fails on unexpected value patterns; what before changing rules?	**Profile the source data**
☐	**Q3**	Easy	New supplier dataset; need completeness and distribution evidence	**Profile the source data first**
☐	**Q4**	Easy	Avoid designing on assumptions about source quality	**Profile the source data**
☐	**Q101**	Medium	Retail loyalty feed, no reliable data dictionary	**Run profiling first; use findings to design validation/cleansing**
☐	**Q147**	Hard	Same field name, different value conventions across sources (governance)	**Profile representative sources, document patterns, base SHARED rules on findings**

2. Error Quarantine & Restart 2 Qs (1 Medium · 1 Hard)

RULE Never stop the whole run for a few invalid records. Route failed rows to a separate error/quarantine flow, let valid rows continue and commit, keep rejects traceable, and reprocess them in a controlled way after correction (without duplicating accepted rows).

HOOK *Bad rows go to the side room; good rows keep moving.*

TRAPS Stop on first invalid record · discard rejects with no trace · load invalid rows into production and flag later · rerun the complete source.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q128**	Medium	Invalid records with missing mandatory fields; stopping is unacceptable	**Route invalid rows to separate error/quarantine flow while valid rows continue**
☐	**Q142**	Hard	Pipeline stops on first bad row and reruns from start; no duplicates on rerun	**Quarantine path + commit valid processing + controlled reprocessing of corrected rejects**

3. Hierarchical Data (JSON Flattening) 2 Qs (1 Medium · 1 Hard)

RULE Nested JSON/arrays must be flattened into an analysis-friendly relational shape while PRESERVING parent-child relationships (carry parent identifiers to every child record).

HOOK *Flatten, but keep the family tree (parent IDs).*

TRAPS Discard nested elements · treat nested arrays as unrelated files · flatten without parent identifiers · store nested objects with no relationship keys · use a Sorter to flatten.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q116**	Medium	JSON with nested sessions/devices/events; raw JSON hard to analyse	**Flatten into relational shape while preserving relationships**
☐	**Q139**	Hard	Claim JSON with repeating procedure arrays; naive flatten creates orphans	**Flatten with preserved parent identifiers so children link to the right parent**

Group B | Mapping Transformations

4. Expression Transformation 5 Qs (4 Easy · 1 Medium)

RULE Row-level calculations, conversions, standardisation and derived columns (NET_AMOUNT, phone formatting, date conversion, status flag). No grouping, no joining. One clear place for derivation logic instead of duplicated calculations.

HOOK *E = Each row, new or changed field.*

TRAPS Sorter, Router, Union or Aggregator used for formulas.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q5**	Easy	NET_AMOUNT from quantity, price, discount	**Expression**
☐	**Q6**	Easy	Standardise phone number and derive a status flag	**Expression**
☐	**Q7**	Easy	New output column from several incoming fields, no grouping/joining	**Expression**
☐	**Q8**	Easy	Timestamp to reporting date plus derived business column	**Expression**
☐	**Q130**	Medium	Duplicated derivations across branches; want one clear place	**Expression to calculate and standardise the derived fields**

5. Filter Transformation 4 Qs (4 Easy)

RULE Keeps only rows that satisfy a row-level condition; everything else is dropped (STATUS = APPROVED, active providers, current business date).

HOOK *Filter = Fewer rows.*

TRAPS Aggregator, Joiner or Union used to remove rows.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q9**	Easy	Only STATUS = APPROVED continue to target	**Filter**
☐	**Q10**	Easy	Only active provider rows needed downstream	**Filter**
☐	**Q11**	Easy	Stop low-value records using a single row-level condition	**Filter**
☐	**Q12**	Easy	Target needs only current-business-date records	**Filter**

6. Joiner Transformation 4 Qs (4 Easy)

RULE Combines rows from TWO pipelines using a join condition (CUSTOMER_ID, ORDER_ID). Adds columns from matching rows. Different from Union (stacks rows) and Lookup (fetches from a reference object).

HOOK *Joiner = Join columns side by side on a key.*

TRAPS Router, Rank, Sequence Generator or Union used to combine on a key.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q13**	Easy	Customers + support tickets by CUSTOMER_ID	**Joiner**
☐	**Q14**	Easy	Two datasets from different sources matched on common key	**Joiner**
☐	**Q15**	Easy	Orders + shipments combined when ORDER_ID matches	**Joiner**
☐	**Q16**	Easy	Combine two streams on explicit equality, not append	**Joiner**

7. Aggregator Transformation 5 Qs (4 Easy · 1 Medium)

RULE Grouped SUM, COUNT, MIN, MAX, AVG using group-by fields (per subscriber per day, per branch, per policy type). Replaces duplicated expression branches used to compute totals.

HOOK *Aggregator = All rows in, one row per group out.*

TRAPS Lookup, Router (one group per total), Sorter or Rank to compute totals.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q17**	Easy	Total call duration per subscriber per day	**Aggregator**

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q18**	Easy	One row per branch with COUNT and AVG	**Aggregator**
☐	**Q19**	Easy	Claim amounts summarised by policy type	**Aggregator**
☐	**Q20**	Easy	Maximum transaction amount per customer	**Aggregator**
☐	**Q104**	Medium	SUM/COUNT/AVG per subscriber and date; simplify duplicated branches	**Aggregator grouped by subscriber and date**

8. Router Transformation 5 Qs (4 Easy · 1 Medium)

RULE One input, several output groups with different conditions, evaluated centrally once (Domestic/International/Review; Accepted/Rejected/Manual Review). Replaces repeated Filter branches.

HOOK *Router = Route to many named groups.*

TRAPS Multiple Aggregators, Union with one condition per output, Sorter, Joiner for branching.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q21**	Easy	Domestic / International / Review paths in one mapping	**Router**
☐	**Q22**	Easy	Accepted / Rejected / Manual Review outputs	**Router**
☐	**Q23**	Easy	One row evaluated against several conditions	**Router**
☐	**Q24**	Easy	Duplicated filter branches per category; centralise	**Router**
☐	**Q105**	Medium	Deposit / Withdrawal / Transfer / Review paths	**One Router with separate output-group conditions**

9. Lookup Transformation & Caching 7 Qs (4 Easy · 1 Medium · 2 Hard)

RULE Retrieves attributes from a reference object by key (PRODUCT_ID to CATEGORY/BRAND; branch code to region) without combining two pipelines. Small, stable reference: use a cached lookup. Large daily-changing reference with huge fact volume: batch-scoped cache refreshed each run. Reference grows to millions or changes often: re-evaluate and consider pushdown / DB-side join instead of assuming the full cache is still best.

HOOK *Lookup = look up a value in a reference object.*

TRAPS Union, Rank or Command task to fetch reference data · uncached lookup per row · re-reading the source for each row · a persistent cache that never refreshes · keeping a huge full cache regardless of growth.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q25**	Easy	PRODUCT_ID needs CATEGORY and BRAND from a reference table	**Lookup**
☐	**Q26**	Easy	Translate branch code to region name	**Lookup**
☐	**Q27**	Easy	Enrich policy with risk class from small reference dataset	**Lookup**
☐	**Q28**	Easy	Retrieve columns by business key without combining pipelines	**Lookup**
☐	**Q106**	Medium	Millions of rows, small rarely-changing reference, repeated DB calls	**Appropriately CACHED lookup**
☐	**Q134**	Hard	200K-row lookup changes daily; tens of millions of fact rows; stale data unacceptable	**Lookup cache REFRESHED EACH BATCH (local resolution, current data)**
☐	**Q150**	Hard	Reference grew to millions and updates often; full cache too heavy	**Evaluate pushdown / database-side join or enrichment**

10. Sorter Transformation 5 Qs (4 Easy · 1 Medium)

RULE Orders rows ascending/descending by sort keys (invoice date desc; region then date; customer number for legacy file). Used when ORDER matters, not for filtering, grouping or joining.

HOOK *Sorter = Sequence only.*

TRAPS Rank (sorts only), Aggregator, Lookup, Router to control order.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q29**	Easy	Invoice rows ordered by INVOICE_DATE descending	**Sorter**

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q30**	Easy	Order by region, then transaction date	**Sorter**
☐	**Q31**	Easy	Control row order, not filter/group/join	**Sorter**
☐	**Q32**	Easy	File in customer-number order for legacy consumer	**Sorter before the target**
☐	**Q122**	Medium	Legacy export in customer and shipment-date order	**Sorter with the required sort keys**

11. Union Transformation 5 Qs (4 Easy · 1 Medium)

RULE Appends rows from multiple compatible-structure pipelines into one stream. No key matching (North+South feeds, monthly files, two claim feeds).

HOOK *Union = stack streams Up-and-down (append).*

TRAPS Joiner without a join condition or with a fabricated key · Router to concatenate · Lookup to stack rows · Rank to merge.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q33**	Easy	North and South region feeds with same structure	**Union**
☐	**Q34**	Easy	Append two compatible datasets, no key match	**Union**
☐	**Q35**	Easy	Monthly files via separate paths into one stream	**Union**
☐	**Q36**	Easy	Merge pipelines vertically, not combine columns	**Union**
☐	**Q123**	Medium	Two claim feeds, same structure, no key match requirement	**Union to combine compatible row streams**

12. Rank Transformation 5 Qs (4 Easy · 1 Medium)

RULE Selects the top or bottom N rows by a ranking measure, optionally per group, and keeps full row detail (top 5 orders per region, lowest 3 defect rates, top 10 products by revenue). Sorting alone would still pass every row.

HOOK *Rank = Top or bottom N.*

TRAPS Sorter alone · Aggregator (loses detail) · Router · Lookup to determine rank.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q37**	Easy	Five highest-value orders per region	**Rank**
☐	**Q38**	Easy	Only the lowest three defect-rate records	**Rank**
☐	**Q39**	Easy	Top products by revenue, keep row detail	**Rank**
☐	**Q40**	Easy	Limited number of best/worst records by a measure	**Rank**
☐	**Q124**	Medium	Top ten products by revenue in each region; sorting alone passes all rows	**Rank configured for required top records**

Group C | Reuse & Configuration

13. Mapplets & Reusable Logic 7 Qs (4 Easy · 1 Medium · 2 Hard)

RULE Repeated transformation sequences go into a reusable MAPPLET so rule changes happen once. Governed reuse: put STABLE common rules in the mapplet and keep client-specific logic OUTSIDE it. When a change could affect existing consumers, release a managed NEW VERSION and migrate consumers deliberately after validation.

HOOK *Write once, reuse many (and version it).*

TRAPS Copy transformations into each mapping · hardcode logic in mapping tasks · a separate Secure Agent per copy · one giant global mapplet with all client rules · edit the shared object in place without testing · remove reuse and hardcode locally.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q41**	Easy	Five mappings repeat the same standardisation/validation	**Reusable mapplet**
☐	**Q42**	Easy	Shared cleansing rule changes often; must stay consistent	**Reusable mapplet**
☐	**Q43**	Easy	One maintained component instead of copies	**Reusable mapplet**

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q44**	Easy	Reusable sequence; each mapping keeps own source/target	**Reusable mapplet**
☐	**Q102**	Medium	Client pipelines repeat trim/standardise/validate; rule change edits many mappings	**Move shared sequence into reusable mapplet logic**
☐	**Q135**	Hard	Central DQ rules plus client-specific flexibility; copies caused drift	**Stable common rules in mapplet; client-specific logic outside the shared component**
☐	**Q151**	Hard	Shared cleansing used by 20 mappings; change could alter history for old clients	**Managed new version; migrate consumers deliberately after validation**

14. Mapping vs Mapping Task 5 Qs (4 Easy · 1 Medium)

RULE The MAPPING holds source-to-target transformation design. The MAPPING TASK is the runnable asset with runtime settings, parameter values, runtime environment and schedule. One mapping can have many tasks.

HOOK *Mapping = blueprint. Mapping task = run ticket.*

TRAPS All transformation design in the task · mapping used for scheduling · connection object instead of a task · copying the mapping per schedule.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q45**	Easy	Design complete, need a runnable asset with runtime settings	**Transformation design in mapping; mapping task for runnable execution settings**
☐	**Q46**	Easy	Separate design-time logic from execution config	**Mapping = design; mapping task = runtime config**
☐	**Q47**	Easy	Logic reusable; schedules/runtime values vary per run	**Mapping for design + mapping task for execution settings**
☐	**Q48**	Easy	Change runtime properties without redesign	**Mapping (design) + mapping task (runtime)**
☐	**Q120**	Medium	One validated mapping, many jobs with different parameters/schedules	**Reuse the mapping; create several mapping task configurations**

15. Parameterization 7 Qs (4 Easy · 1 Medium · 2 Hard)

RULE Parameters let paths, file names, table names, schemas and connections change at runtime or per environment without rewriting or duplicating the mapping. One transformation design for dev/test/prod and for many tenants; secrets are never hardcoded; approved config supplied at deployment/execution.

HOOK *Same logic, changing values.*

TRAPS Duplicate or clone mapping per environment/tenant · hardcode values in expressions · new Secure Agent per value · Sorter storing tenant names.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q61**	Easy	Dev/test/prod file paths differ; same logic	**Use parameters**
☐	**Q62**	Easy	One mapping loads different target table by context	**Use parameters**
☐	**Q63**	Easy	Change source file name at runtime without republishing	**Use parameters**
☐	**Q64**	Easy	Reusable pipeline with environment-specific values	**Use parameters**
☐	**Q107**	Medium	Separate mapping copies per environment (schemas, paths, connections)	**Parameterize environment-specific values; supply at runtime**
☐	**Q137**	Hard	Promotion with manual edits and sensitive values	**Parameterize env properties; supply approved runtime configuration at deployment/execution**
☐	**Q145**	Hard	Per-tenant tables/paths; frequent tenant onboarding	**Parameters for tenant-specific object names/paths; one transformation design**

16. Connections 6 Qs (4 Easy · 1 Medium · 1 Hard)

RULE A CONNECTION holds endpoint (host, service) and authentication properties for a source/target. Create once, reuse in many mappings/tasks. Credentials never live in expressions, notes or local files. An endpoint change is fixed in one place.

HOOK *Connection = the stored key to a system.*

TRAPS Endpoint in taskflow decision step · mapplet as credential store · monitoring dashboard to define access · hardcode in expressions · credentials in notes/branch labels · duplicate mappings when endpoint changes.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q57**	Easy	New Oracle source: host, service, user, authentication	**Configure a connection**
☐	**Q58**	Easy	Same target used by several mappings	**Connection defines access centrally**
☐	**Q59**	Easy	Reusable endpoint + login definition for cloud DB	**Create a connection**
☐	**Q60**	Easy	Access details needed before selecting source object	**Connection**
☐	**Q119**	Medium	Several mappings read same cloud DB; endpoint repeated	**Create and reuse one connection**
☐	**Q148**	Hard	Credentials in mapping notes/local files; security wants central control	**Centrally managed IDMC connection objects referenced from mappings/tasks**

Group D | Runtime Infrastructure

17. Secure Agent 6 Qs (4 Easy · 1 Medium · 1 Hard)

RULE The Secure Agent executes jobs and provides connectivity between IDMC and configured cloud or on-premises systems, so protected databases are never exposed publicly. For resilience, run MULTIPLE Secure Agents in one runtime environment (no single point of failure, no redesign).

HOOK *Secure Agent = the engine that lives inside your network.*

TRAPS Taskflow, mapplet or dashboard as runtime connectivity · expose the DB publicly · duplicate mappings per host · decision step as redundancy · one Secure Agent per copy of logic.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q49**	Easy	CDI mapping must read on-premises DB not exposed to cloud	**Secure Agent**
☐	**Q50**	Easy	Need runtime component near enterprise data source	**Secure Agent**
☐	**Q51**	Easy	Jobs access internal systems while keeping cloud design	**Secure Agent**
☐	**Q52**	Easy	Component bridging IDMC and data systems during execution	**Secure Agent**
☐	**Q117**	Medium	Factory DB only reachable inside corporate network	**Secure Agent in the appropriate runtime environment**
☐	**Q136**	Hard	Single Secure Agent is a runtime single point of failure	**Runtime environment with MULTIPLE Secure Agents**

18. Runtime Environment 5 Qs (4 Easy · 1 Medium)

RULE A RUNTIME ENVIRONMENT identifies the runtime resources (one or more Secure Agents) that execute jobs. Assign the mapping task to the environment that holds the approved agents.

HOOK *Runtime environment = the agent group you pick for a task.*

TRAPS Project folder, connection object or mapplet as execution container · storing agent names in an Expression.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q53**	Easy	Assign mapping task to the right group of Secure Agents	**Runtime environment**
☐	**Q54**	Easy	Use a defined set of runtime resources, not any agent	**Runtime environment**
☐	**Q55**	Easy	Specify where the task will execute	**Runtime environment**
☐	**Q56**	Easy	Associate several managed agents with job execution	**Runtime environment**
☐	**Q118**	Medium	Task must run only on finance-approved agents	**Assign task to runtime environment containing approved agents**

Group E | Taskflow Orchestration

19. Sequential Taskflow 4 Qs (4 Easy)

RULE Use sequential execution when each step depends on completion/output of the previous one (ingest, validate, load; downstream consumes prior file).

HOOK *Sequential = one after another because each needs the last.*

TRAPS Parallel execution of dependent steps · Router or Union to orchestrate tasks.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q65**	Easy	Ingest, validate, load; each needs the previous	**Sequential taskflow**
☐	**Q66**	Easy	Three mappings with strict dependencies, no overlap	**Sequential taskflow**
☐	**Q67**	Easy	Downstream task consumes file from prior task	**Sequential taskflow**
☐	**Q68**	Easy	Fixed order; each step needs preceding results	**Sequential taskflow**

20. Parallel Taskflow 8 Qs (4 Easy · 2 Medium · 2 Hard)

RULE Independent tasks run in parallel paths to cut elapsed time. A dependent final step (consolidation/aggregate) is gated until all required branches complete. Real designs are HYBRID: shared prerequisites sequential, independent branches parallel, final step after all branches. Same principle applies to independent CAI service calls (parallel, then join before consolidation).

HOOK *Parallel for independents, then GATE the dependent step.*

TRAPS Keep everything sequential · run consolidation in parallel with its inputs · remove validation dependencies · Joiner/Lookup/parameter to simulate parallelism · scale compute only · duplicate ingestion in each branch · cache old responses.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q69**	Easy	Four regional mappings, no dependencies, tight window	**Parallel execution paths**
☐	**Q70**	Easy	Two independent validations wait for each other	**Parallel execution paths**
☐	**Q71**	Easy	Several independent branches can run together	**Parallel execution paths**
☐	**Q72**	Easy	Shorten run, tasks independent, no mapping change	**Parallel execution paths**
☐	**Q109**	Medium	Five independent regional pipelines run sequentially	**Run independent regional branches in parallel**
☐	**Q129**	Medium	CAI process calls several independent services before final consolidation	**Run independent calls in parallel; join before consolidation**
☐	**Q133**	Hard	8 regional mappings + shared validation + final consolidation	**Parallel regional mappings, then GATE consolidation on required branches**
☐	**Q144**	Hard	Nightly: shared prerequisites, regional transforms, final aggregate; SLA missed by 40 min	**Prerequisites sequential, regionals parallel, aggregate after all branches complete**

21. Decision Step (Conditional Branching) 6 Qs (4 Easy · 1 Medium · 1 Hard)

RULE A decision/conditional step evaluates a value from an earlier step and sends the taskflow down the correct branch (PASS to load / FAIL to alert; zero vs non-zero rows; success / warning / critical with distinct branches).

HOOK *Decision = fork in the road based on a runtime result.*

TRAPS Sorter, Union or Joiner to choose the path · single fixed sequential path · run load and alert in parallel every time · every branch in parallel regardless of result.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q73**	Easy	Load only if validation = PASS, otherwise alert	**Decision/conditional step**
☐	**Q74**	Easy	Different path for zero-row vs non-zero input	**Decision/conditional step**
☐	**Q75**	Easy	Next task depends on a value returned earlier	**Decision/conditional step**
☐	**Q76**	Easy	Different behaviour for success, warning, exception	**Decision/conditional step**
☐	**Q108**	Medium	Validation fails: stop loading and run alert; success: continue	**Decision step branching on validation outcome**

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q149**	Hard	Warnings: load + warn; critical: stop + remediation; native orchestration	**Decision logic with distinct branches for pass, warning, critical**

22. Fault Handling 7 Qs (4 Easy · 1 Medium · 2 Hard)

RULE Configure fault handling so failures are captured and managed (retry, alternate path, notification) instead of uncontrolled termination. Be GRANULAR: critical services stop/recover; non-critical ones are isolated with retry/alternate handling. For multi-step recovery, combine fault handling with idempotent/checkpointed steps so restart resumes from a safe known point.

HOOK *Plan the failure path before it happens.*

TRAPS Ignore errors / mark success · duplicate mappings per error · Sorter or Union as recovery · one ignore-error rule for everything · stop everything for a non-critical failure · remove fault handling · rerun every step blindly · delete monitoring history.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q77**	Easy	External service fails temporarily; need controlled response	**Fault handling**
☐	**Q78**	Easy	Record failures and run a defined recovery path	**Fault handling**
☐	**Q79**	Easy	Predictable behaviour on step failure, not manual recovery	**Fault handling**
☐	**Q80**	Easy	Distinguish success from failure and recover	**Fault handling**
☐	**Q110**	Medium	CAI workflow: one failed API call stops everything	**Fault handling with appropriate retry/recovery path**
☐	**Q138**	Hard	Payment flow: notification outage must not roll back ledger; identity/risk failure must stop	**Fault handling PER service path: stop/recover for critical; isolated retry/alternate for non-critical**
☐	**Q159**	Hard	Ingest OK, transform fails; no double ingestion; clear restart point	**Fault handling + idempotent/checkpointed steps; resume from safe known point**

23. Command & Event Steps (Triggers/Listeners) 6 Qs (4 Easy · 2 Medium)

RULE Use a taskflow COMMAND step for system-level/shell actions outside the mapping. Use TRIGGER/LISTENER capability to start a taskflow when an event arrives rather than only on a schedule.

HOOK *Command = do a system action. Event = start when something happens.*

TRAPS Aggregator, Joiner, Lookup or Sorter executing commands or waiting for events · monthly polling · manual run with no trigger.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q81**	Easy	Taskflow must invoke a shell-level utility	**Taskflow command capability**
☐	**Q82**	Easy	Workflow begins when an event signal arrives, not only at a fixed time	**Event capability (trigger/listener)**
☐	**Q83**	Easy	React to an incoming event, then start downstream steps	**Event-driven taskflow capability**
☐	**Q84**	Easy	System command outside the mapping itself	**Taskflow command step**
☐	**Q126**	Medium	Run a system utility after export completes	**Taskflow command step**
☐	**Q127**	Medium	Process starts on an unpredictable application event	**Taskflow trigger/listener for the event**

Group F | Data Movement Patterns

24. Change Data Capture (CDC) / Incremental 6 Qs (4 Easy · 1 Medium · 1 Hard)

RULE CDC/incremental processing handles only inserts, updates and deletes since the last cycle, avoiding full reloads. For recovery, persist and advance the checkpoint/watermark ONLY AFTER successful processing and restart from the last committed checkpoint (no gaps, no duplicates).

HOOK *Only carry what changed, and only move the bookmark after success.*

TRAPS Full reload every cycle · Sorter or result caching to detect changes · advance checkpoint before processing · restart from current clock time · full reload after every failure.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q85**	Easy	Reloading millions of unchanged rows each run	**CDC / incremental processing**
☐	**Q86**	Easy	Process only records modified since last successful run	**CDC / incremental processing**
☐	**Q87**	Easy	Capture inserts, updates, deletes without re-reading source	**CDC / incremental processing**
☐	**Q88**	Easy	Volume grows but only a small % changes	**CDC / incremental processing**
☐	**Q111**	Medium	Order statuses change daily but full table reloaded	**Incremental change-data processing**
☐	**Q132**	Hard	Run failed before watermark advanced; avoid duplicates or gaps	**Advance checkpoint only after successful processing; restart from last committed checkpoint**

25. Batch vs Real-Time Ingestion 6 Qs (4 Easy · 1 Medium · 1 Hard)

RULE Batch = scheduled grouped loads that tolerate latency (nightly files, settlement, reconciliation, monthly reference files). Real-time = continuously arriving events needing low latency (fraud, authorization events). Match each workload to its own latency requirement.

HOOK *Latency decides: scheduled = batch, continuous = real-time.*

TRAPS Batch for continuous low-latency events · real-time for monthly/nightly files · treating both as identical · one pattern for all · manual ingestion.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q93**	Easy	Nightly file drop vs continuous fraud events	**Batch for scheduled loads; real-time for continuous low latency**
☐	**Q94**	Easy	Once-per-day load for one feed, immediate for another	**Batch for scheduled grouped; real-time for continuous**
☐	**Q95**	Easy	Periodic grouped vs continuous event processing	**Batch = scheduled grouped; real-time = continuous low latency**
☐	**Q96**	Easy	Monthly reference files vs continuous transactions	**Batch for monthly; real-time for continuous**
☐	**Q112**	Medium	Midnight settlement files vs continuous authorizations	**Batch for settlement; real-time for authorization events**
☐	**Q146**	Hard	Continuous authorizations vs nightly reconciliation files	**Real-time for authorizations; scheduled batch for reconciliation**

Group G | Performance & Observability

26. SQL Pushdown 10 Qs (4 Easy · 2 Medium · 4 Hard)

RULE Push compatible filtering, projection, joins, aggregations to the database so processing happens near the data and less data moves. Push source predicates/columns to the source. If something blocks pushdown: (a) REFACTOR only the incompatible expressions, (b) pre-derive a simple compatible join key instead of a complex join expression, (c) use PARTIAL

pushdown: push what is compatible and keep only the incompatible rule in the engine. Never change business semantics to force pushdown.

HOOK *Take the work to the data, not the data to the work.*

TRAPS Move more data into the engine · disable pushdown · scale the Secure Agent instead · force pushdown without compatibility check · change the business rule · extract all history then filter later · stage every result to files · Router+Union instead of join.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q89**	Easy	Large dataset moved to engine before filtering	**Push compatible work to the database**
☐	**Q90**	Easy	Snowflake compute idle while Secure Agent does joins/aggregations	**Push compatible work to the database**
☐	**Q91**	Easy	Filtering/aggregation where data already resides	**Pushdown**
☐	**Q92**	Easy	Use DB processing power, same business logic	**Pushdown**
☐	**Q103**	Medium	Large shipment/customer join inside engine; simple compatible keys	**Pushdown-compatible join so the DB performs it**
☐	**Q121**	Medium	Hundreds of columns, years of history; only recent rows/few fields used	**Apply compatible filtering/projection close to source via pushdown**
☐	**Q131**	Hard	Some expressions use functions incompatible with DB execution	**Refactor ONLY the incompatible expressions so eligible operations push down**
☐	**Q141**	Hard	Custom expression in join condition keeps join in engine	**Derive compatible join key BEFORE the join; simple DB-compatible condition**
☐	**Q154**	Hard	Five years in source; only yesterday needed; filter done in engine	**Push the date predicate to the source/database**
☐	**Q160**	Hard	One rule is not pushdown-capable; exact semantics must be kept	**Partial pushdown: push compatible ops, keep incompatible rule in engine**

27. Monitoring & Performance Diagnosis 4 Qs (2 Medium · 2 Hard)

RULE Use IDMC monitoring and execution logs (run status, step-level behaviour, lookup activity, data volume) to find the real bottleneck, then apply a targeted optimisation (pushdown, lookup strategy, less data movement). Step-level taskflow monitoring isolates slow or failing steps. Never just add compute blindly.

HOOK *Evidence first, targeted fix second.*

TRAPS Increase warehouse/compute without evidence · disable logging/monitoring · duplicate mappings · rewrite everything unanalysed · record only final status · raise timeouts without measuring.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q113**	Medium	CDI jobs fail intermittently; engineers search runs manually	**IDMC monitoring and execution logs**
☐	**Q125**	Medium	Mapping slowed as volume grew; heavy engine work, repeated reference access	**Review logs and design; optimise pushdown, lookup strategy, data movement**
☐	**Q140**	Hard	Unpredictable spikes not tied to row count; compute raised reactively	**Correlate logs, step-level behaviour, lookup activity, data volume; then targeted optimisation**
☐	**Q152**	Hard	Taskflow with 5 tasks + 2 external calls; final status hides the slow step	**Detailed taskflow/run monitoring and step-level logs**

28. Snowflake Context Questions 4 Qs (4 Hard)

RULE These four hard questions are source-material facts about Snowflake: concurrency = multi-cluster warehouse; idle cost = auto-suspend (with resume); result-cache reuse only while the query is eligible AND underlying data is unchanged; poor micro-partition pruning = clustering on frequently filtered columns.

HOOK *Multi = many users. Suspend = save money. Cache = same query, same data. Cluster = prune.*

TRAPS Result caching as the only concurrency strategy · sequential schedule for everything · always-on warehouses · changing mappings/connections for cost · assuming cache survives data change · using clustering for billing or stale results · Router/taskflow parallelism to fix pruning.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q155**	Hard	Loads + dashboards queue at peak; bigger warehouse not enough	**Multi-cluster warehouse behaviour**

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q156**	Hard	Warehouses idle between scheduled jobs; cost continues	**Automatic suspension with resume as needed**
☐	**Q157**	Hard	Identical queries fast, slow after daily data change	**Result-cache reuse only while query is eligible and data unchanged**
☐	**Q158**	Hard	Large table filtered by date/region scans too many micro-partitions	**Appropriate clustering on frequently filtered columns**

Group H | CAI & CAI-CDI Integration

29. CAI Connectors & CAI-CDI Integration 8 Qs (4 Easy · 2 Medium · 2 Hard)

RULE Use CAI SERVICE CONNECTORS for external APIs/services (reusable, centrally maintained endpoint, auth and error handling; process-specific logic stays in each process). CAI can invoke CDI mappings inside the same orchestration. For long data jobs, trigger CDI ASYNCHRONOUSLY and return a TRACKABLE status instead of blocking the API call.

HOOK *CAI talks to the outside world and calls CDI; no human in the middle.*

TRAPS Keep CAI and CDI isolated with manual handoff · Lookup/mapping transformations as API client · monitoring logs to trigger jobs · run API manually outside IDMC · hardcode endpoint in each process · copy whole process per use case · CDI mapplets as API mechanism · synchronous wait until all data steps finish · return success with no status tracking.

✓	Q#	Level	Scenario cue (what the question is really asking)	Answer to remember
☐	**Q97**	Easy	API process calls external service then starts CDI pipeline automatically	**CAI service connectors + integrated invocation**
☐	**Q98**	Easy	CAI workflow needs reusable app connectivity and a downstream CDI mapping	**CAI service connectors + integrated invocation**
☐	**Q99**	Easy	One orchestrated process across APIs and data jobs	**CAI service connectors + integrated invocation**
☐	**Q100**	Easy	Service request must trigger a CDI mapping in same orchestration	**CAI service connectors + integrated invocation**
☐	**Q114**	Medium	Booking workflow calls partner pricing API	**CAI service connector in the process**
☐	**Q115**	Medium	CAI API validates input then should start Snowflake-loading CDI mapping (no manual start)	**Invoke CDI from CAI orchestration via supported integration**
☐	**Q143**	Hard	API blocks until pipeline ends and times out; async acceptable	**Trigger CDI asynchronously; return trackable process status**
☐	**Q153**	Hard	Many CAI processes call same verification API, inconsistent error handling	**Reusable service connector/configuration; keep workflow-specific logic in each process**

5. Revision Plan & Final Checklist

Step	What to do
Pass 1: Understand (about 45 min)	Read Section 2 (cheat sheet) and Section 3.1 (look-alikes). Then read only the RULE and HOOK lines of each topic in Section 4.
Pass 2: Active recall (about 60 min)	Go topic by topic. Cover the Answer column, answer from the cue alone, and tick ☐only when correct. Easy topics should go fast because the twin questions repeat.
Pass 3: Hard & Medium focus (about 45 min)	Re-attempt every unticked question first. Then go through the Hard-tier signatures (3.3). These 60 questions carry 120 marks.
Day-before / last 15 min	Re-read the cheat sheet, the "always wrong" blacklist (3.2), and the Snowflake hooks (Q155–Q158).

Three questions to ask on every MCQ

1. Which native IDMC feature is built for exactly this job? (right transformation / taskflow step / runtime object)
2. Does the option keep the business logic unchanged while removing the stated pain? (reject anything that changes semantics, hardcodes, duplicates or hides errors)
3. Is it evidence-based and targeted? (profile first, monitor first, push down only what is compatible, checkpoint only after success)

Coverage confirmation

All 160 questions (Q1–Q160) are mapped to exactly one topic: 100 Easy, 30 Medium, 30 Hard across 29 topics in 8 groups. Section 1 lists every question number per topic; Section 4 has a tick box for each one.